

オープンデータとは

- 政府・自治体が **誰でも自由に使える形式** で公開するデータ
- 商用・非商用問わず **二次利用可能**
- 機械可読（CSV, JSON, API）で提供

① 意義

- **透明性**：行政の活動を市民が検証できる
- **イノベーション**：民間がデータを活用してサービス開発
- **効率化**：データに基づく政策立案（**EBPM: Evidence-Based Policy Making**）

身近なオープンデータ活用例

サービス	使用データ
乗換案内アプリ	公共交通の時刻表・遅延情報
防災アプリ	避難所・ハザードマップ
不動産サイト	地価公示・用途地域
天気予報	気象庁の観測データ

- **すでに使っている** — 意識していないだけ
- 自分でもAPIを叩けば **同じデータ** が手に入る



概要

- **東京都オープンデータカタログサイト** の一部データをAPI提供
- ユーザ登録 **不要** — 誰でもすぐに利用可能
- 掲載データ約54,000件のうち、CSV約35,000件がAPI対応

項目	内容
認証	不要
レート制限	明示なし（常識的な範囲で）
データ形式	JSON / XML
CORS	対応済み

6

Wikipedia APIとの違い

	Wikipedia API	東京都API
HTTPメソッド	GET	POST
データ内容	百科事典・統計	行政・地理・施設
更新頻度	リアルタイム～日次	不定期（年次も多い）

⚠ 重要：POSTでアクセス

- Wikipedia API は **requests.get()**
- 東京都API は **requests.post()**
- 間違えると動かない

7

APIの構造

8

基本URL

```
1 https://service.api.metro.tokyo.lg.jp/api/{apiId}/{format}
```

- **{apiId}** — データセットごとに異なるID
- **{format}** — **json** または **xml**

例：東京都人口の推移（JSON）

```
1 https://service.api.metro.tokyo.lg.jp/api/t000003d2000000236-c054eb26d9e9721642bef78c7c717e76-0/json
```

9

APIのIDを探す

- [API検索ページ](#) でキーワード検索
- 欲しいデータの apild をコピーして使う

💡 探し方のコツ

- 「避難所」「駐輪場」「人口」などキーワードで検索
- 区市町村名で絞り込み可能
- 詳細ページで apild と列名を確認

10

パラメータ

パラメータ	説明	例
<code>limit</code>	取得レコード数の上限	<code>params = {"limit": 200}</code>
<code>offset</code>	取得開始位置	<code>params = {"offset": 50}</code>

- デフォルトは **100件** — 全件欲しいときは `limit` を大きく設定
- 取得する **列の絞り込み** は、URLではなく **POSTのbody** で指定する（後述）

11

基本的な使い方

12

最小テンプレート

```
Code
1 import requests
2
3 api_id = "t000003d2000000236-c054eb26d9e9721642bef78c7c717e76-0"
4 url = f"https://service.api.metro.tokyo.lg.jp/api/{api_id}/json"
5
6 params = {"limit": 200}
7 headers = {"User-Agent": "example.com/lesson"}
8 body = {}
9
10 response = requests.post(url, params=params, headers=headers, json=body)
11 data = response.json()
12
13 print(data["hits"])
```

① `json={}` の意味

- 空のJSONボディを送信している — 東京都APIの仕様上、省略不可
- 省略すると **415**（Unsupported Media Type）が返ってくる — 400番台＝リクエスト側のミス、の実例

13

レスポンス構造

```
1 {
2   "total": 152,
3   "subtotal": 152,
4   "limit": 200,
5   "offset": null,
6   "metadata": {
7     "apiId": "API識別子",
8     "title": "データタイトル",
9     "version": "1.0.7"
10  },
11  "hits": [
12    {"row": 1, "和暦（年次）": "明治5", "西暦（年次）": 1872, "東京都／人口": 859345},
13    {"row": 2, "和暦（年次）": "明治6", "西暦（年次）": 1873, "東京都／人口": 887322},
14    ...
15  ]
16 }
```

- **total** — 全件数
- **metadata** — データセットの情報
- **hits** — **実際のデータ本体**（list of dict）

検索条件を指定する

列を絞る・行を絞る

- **列** の限定は、POSTのbodyに **column** を入れる
 - **行** の絞り込み（期間指定など）は、取得後に **Pythonでフィルタ** するのが確実
- 内包表記の **if** — **Pandas入門** の条件抽出と同じ発想

```
Code
1 # 列を限定して取得
2 body = {
3   "column": ["西暦（年次）", "東京都／人口"]
4 }
5
6 # 取得後に Python でフィルタリング
7 filtered = [r for r in data["hits"]
8             if 1950 <= r["西暦（年次）"] <= 2023]
```

実用サンプル

東京都人口（期間指定）

Code

```
1 import requests
2
3 headers = {"User-Agent": "example.com/lesson"}
4 api_id = "t000003d20000000236-c054eb26d9e9721642bef78c7c717e76-0"
5 url = f"https://service.api.metro.tokyo.lg.jp/api/{api_id}/json"
6
7 params = {"limit": 200}
8 body = {"column": ["西暦（年次）", "東京都／人口"]}
9
10 response = requests.post(url, params=params, headers=headers, json=body)
11 data = response.json()
12
13 # Python でフィルタリング
14 filtered = [r for r in data["hits"]
15             if 1950 <= r["西暦（年次）"] <= 2023]
16
17 print(f"総件数: {len(filtered)}")
18 for record in filtered[:5]:
19     print(f"{record['西暦（年次）']}年: {record['東京都／人口']:,}人")
```

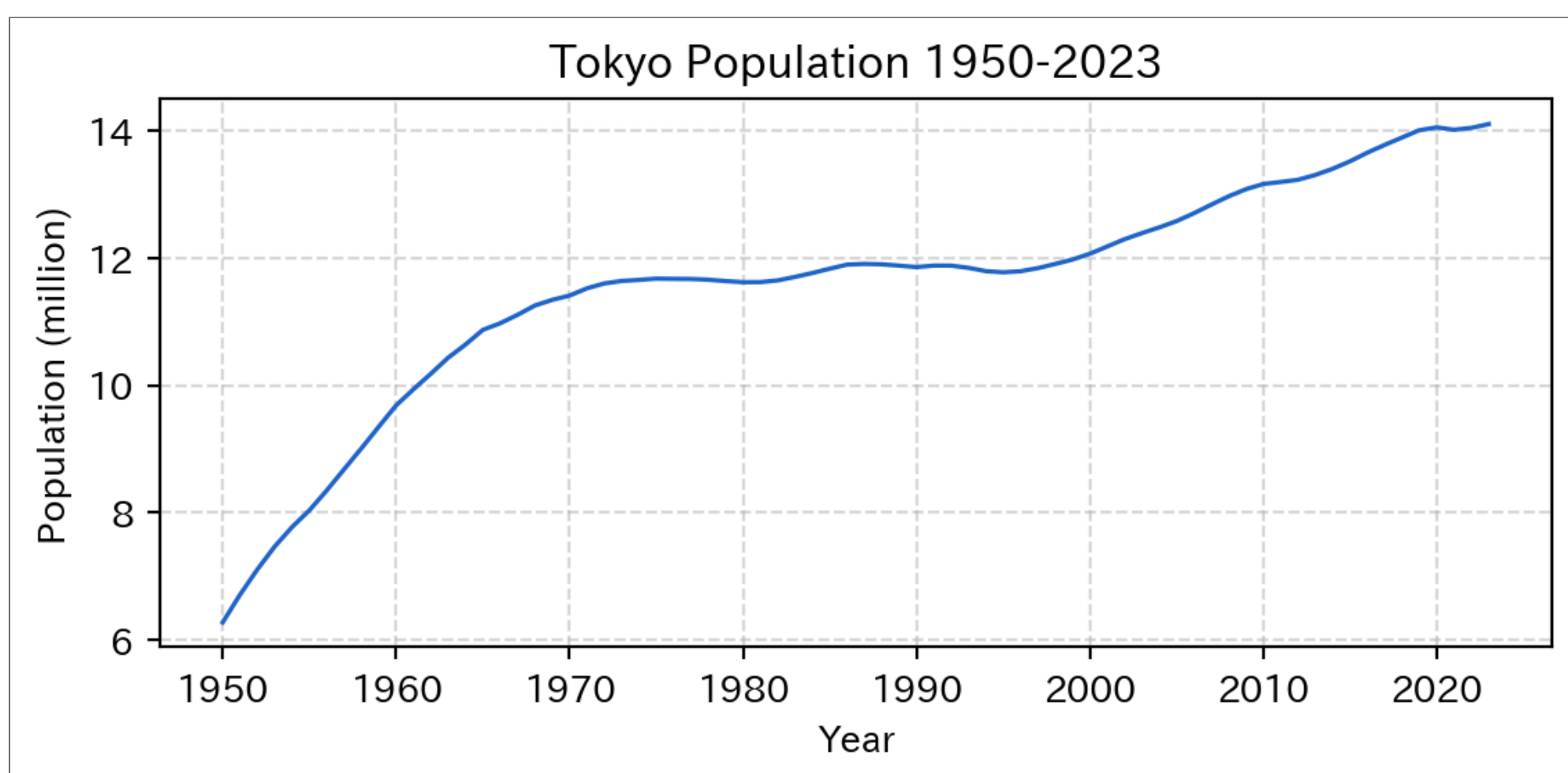
```
1 総件数: 74
2 1950年: 6,277,500人
3 1951年: 6,712,494人
4 1952年: 7,108,749人
5 1953年: 7,468,907人
6 1954年: 7,773,648人
```

18

人口推移の可視化

- さっきの **filtered** から列を取り出して、いつもの **plt.plot()** に渡すだけ

▶ コードを見る



- 戦後復興期から高度経済成長期にかけて **急増**
- 1970年代からしばらく **横ばい**、1990年代後半から再び増加
- このように **まず描く** ことで傾向が見える

19

避難所情報（品川区）

Code

```
1 import requests
2
3 headers = {"User-Agent": "example.com/lesson"}
4 api_id = "t131091d00000000001-3f9091b43cf1caa550e7152b5c2ba34c-0"
5 url = f"https://service.api.metro.tokyo.lg.jp/api/{api_id}/json"
6
7 params = {"limit": 5}
8 body = {}
9
10 response = requests.post(url, params=params, headers=headers, json=body)
11 data = response.json()
12
13 print(f"品川区の避難所情報（総件数: {data['total']}件）")
14
15 for facility in data["hits"]:
16     print("---")
17     for key, value in facility.items():
18         if key != "row":
19             print(f"{key}: {value}")
```

```
1 品川区の避難所情報（総件数: 25件）
2 ---
3 名称: 品川区立第一日野小学校
4 緯度: 35.6234...
5 経度: 139.7312...
6 ---
7 ...
```

避難所リスト（東京都全域）

Code

```
1 import requests
2
3 headers = {"User-Agent": "example.com/lesson"}
4 api_id = "t000003d00000000093-0af9bfa497bc7be6ef83e5bfd1fdcfc6-0"
5 url = f"https://service.api.metro.tokyo.lg.jp/api/{api_id}/json"
6
7 params = {"limit": 5}
8 body = {}
9
10 response = requests.post(url, params=params, headers=headers, json=body)
11 data = response.json()
12
13 print(f"東京都の避難所（総件数: {data['total']}件）")
14 for shelter in data["hits"]:
15     print(f"施設名: {shelter['避難所_施設名称']}")
16     print(f"住所: {shelter['所在地住所']}")
17     print(f"区市町村: {shelter['指定市区町村名']}")
18     print("----")
```

```
1 東京都の避難所（総件数: 2636件）
2 施設名: お茶の水小学校
3 住所: 東京都千代田区神田猿楽町1-1-1
4 区市町村: 千代田区
5 ---
6 施設名: 都立一橋高校
7 住所: 東京都千代田区東神田1-12-13
8 区市町村: 千代田区
9 ---
10 ...
```

都営バス運賃

Code

```
1 import requests
2
3 headers = {"User-Agent": "example.com/lesson"}
4 api_id = "t000018d00000000044-3db21a77b0af1c54b411f79604444ca2-0"
5 url = f"https://service.api.metro.tokyo.lg.jp/api/{api_id}/json"
6 body = {}
7
8 response = requests.post(url, headers=headers, json=body)
9 data = response.json()
10
11 print(f"都営バス運賃情報（総件数: {data['total']}件）")
12
13 for fare_info in data["hits"][:5]:
14     print("---")
15     for key, value in fare_info.items():
16         if key != "row":
17             print(f"{key}: {value}")
```

```
1 都営バス運賃情報（総件数: 4件）
2 ---
3 区分: 一般系統
4 大人: 210円 (IC 210円)
5 小児: 110円 (IC 105円)
6 備考: 急行バス、直行バスを含む。
7 ---
8 ...
```

22

NumPyで統計処理

23

統計分析の例

- **hits** は **list of dict** — 内包表記で列を取り出して **np.array** にすれば、いつもの統計がそのまま使える

Code

```
1 import requests
2 import numpy as np
3
4 headers = {"User-Agent": "example.com/lesson"}
5 api_id = "t000003d2000000236-c054eb26d9e9721642bef78c7c717e76-0"
6 url = f"https://service.api.metro.tokyo.lg.jp/api/{api_id}/json"
7
8 params = {"limit": 200}
9 response = requests.post(url, params=params, headers=headers, json={})
10 data = response.json()
11
12 population = np.array([r["東京都／人口"] for r in data["hits"]])
13
14 print(f"平均: {np.mean(population):,.0f}人")
15 print(f"最大: {np.max(population):,}人")
16 print(f"最小: {np.min(population):,}人")
17 print(f"標準偏差: {np.std(population):,.0f}")
18 print(f"中央値: {np.median(population):,.0f}人")
```

```
1 平均: 7,294,170人
2 最大: 14,099,993人
3 最小: 859,345人
4 標準偏差: 4,567,885
5 中央値: 7,189,875人
```

24

実データは汚れている

- 人口の列は綺麗だったが、同じ表の **増減率(%)** の列を見ると...

Code

```
1 rates_raw = [r["東京都／増減率(%)"] for r in data["hits"]]
2 print(rates_raw[:4])
```

```
1 ['...', '3.26', '5.09', '5.75']
```

- 数値が **文字列** ("**3.26**") で入っていて、データがない年は **"..."**
- 使うには、**"..."** を除いてから **float()** で数値に変換する

Code

```
1 rates = np.array([float(v) for v in rates_raw if v != "..."])
2 print(f"増減率の平均: {rates.mean():.2f}%") # 増減率の平均: 2.10%
```

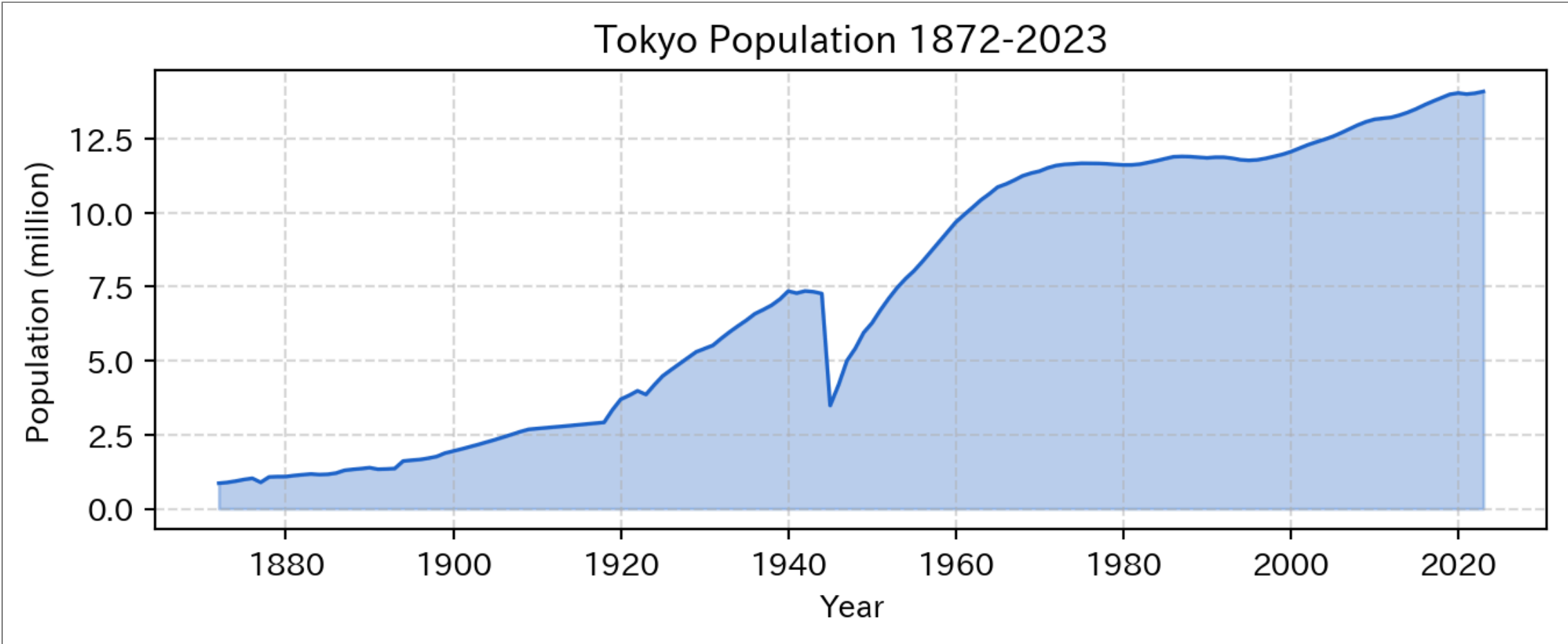
💡 型を確かめてから使う

- 外部APIのデータは **型が揃っていないとは限らない** — 数値のつもりが文字列、欠損が **"..."** などの記号
- 集計の前に **print(type(値))** や **isinstance(値, int)** で確認するクセをつける

25

まずは可視化する

- 数値だけでは傾向がわからない
- データの可視化 の大原則： **Always plot your data first.**



- **右肩上がり** — 150年で約16倍に増加
- **1940年代に急落** — 戦争による疎開・被害
- **2020年代は横ばい** — コロナ禍の2021年には減少もあった

💡 これまでの道具が活きる

- listあらためて の内包表記で抽出
- NumPy入門 の統計関数で集計
- データの可視化 ・ matplotlib入門 の手法で可視化

まとめ

やりたいこと	コード
基本アクセス	<code>requests.post(url, params=..., headers=..., json={})</code>
取得件数指定	<code>params = {"limit": 200}</code>
列の絞り込み	<code>body = {"column": ["列名1", "列名2"]}</code>
行の絞り込み	取得後にPythonでフィルタ（内包表記 + <code>if</code> ）
全件数確認	<code>data["total"]</code>
データ本体	<code>data["hits"]</code> (list of dict)

⚠ Wikipedia API との違い

- **GET** ではなく **POST**
- 空でも `json={}` が必要



演習：避難所データの可視化

任意の区市町村の**避難所データ**を取得し、**件数**を表示せよ。

- （発展）緯度・経度があれば散布図で地図風にプロット

💡 ヒント

- **API検索ページ** [🔗](#) で「避難所」を検索
- 複数の区のデータを比較しても面白い

▶ 解答例を見る（品川区の例 — apildは検索ページで差し替え）

29



演習：人口推移の時系列グラフ

東京都の人口データを**全件**取得し、人口推移を折れ線グラフで描け。

1. 年と人口を取り出して **折れ線グラフ** を描画
2. **最大値・最小値・平均** をタイトルに表示

▶ 解答例を見る

30



演習：2つのデータセットを比較

異なる2つのデータセット（例：2つの区の避難所、戦前と戦後の人口）を取得し、件数や統計量を比較して表示せよ。

- （発展）同じグラフに重ねて描画

▶ 解答例を見る（戦前と戦後の人口を比較する例）

31

アプリケーション例

分野	活用例
防災	避難所マップ、消火栓・防火水槽の位置、地震被害想定
交通	駐輪場検索、バス運賃計算、駅の乗降客数分析
生活	公衆トイレマップ、公共施設案内、子育て支援制度
研究	人口動態分析、地価変動追跡、行政サービス利用調査

33

関連リソース

リソース	URL
オープンデータカタログサイト	portal.data.metro.tokyo.lg.jp
APIカタログサイト	portal.data.metro.tokyo.lg.jp/opendata-api/
API検索ページ	spec.api.metro.tokyo.lg.jp/spec/search
公共交通オープンデータ	odpt.org
デジタル庁 GIF	digital.go.jp

34